Pinecone

Pinecone Vector Database

ARCHITECTURE AND DESIGN PRINCIPLES

Contact us with questions or to schedule a demo.

Pinecone

Executive Summary

Pinecone delivers the vector database for building accurate and performant AI applications at scale. This whitepaper explains the core architectural decisions behind Pinecone's approach to knowledge retrieval. We cover write and query processing, data organization through slabs and compaction, metadata filtering strategies, and distributed systems design. These techniques enable Pinecone to serve billions of vectors with low latency and high accuracy.

Introduction

Pinecone is a vector database built to handle AI workloads at scale. This whitepaper explains how Pinecone's architecture solves fundamental challenges in vector search: fast writes, low-latency queries, and efficient metadata filtering.

We built Pinecone from the ground up to address problems that existing solutions couldn't solve. Traditional vector search systems force you to choose between performance and scale. They require you to tune parameters, reduce data fidelity, or sacrifice write speed for query performance.

Pinecone eliminates these tradeoffs. You can store full-fidelity vectors at any dimension. Writes appear in queries within seconds regardless of index size. Metadata filtering makes queries faster, not slower. The system scales automatically without manual intervention.

This document explains how Pinecone's architecture delivers these capabilities. We focus on the engineering decisions and distributed systems design that enable high performance at scale.

Storage Architecture: LSM-Based Slab System

Pinecone uses a Log-Structured Merge (LSM) tree approach to organize data. We store vectors in immutable files called slabs. This architecture enables fast writes, efficient queries, and background optimization without locking.

What is a “slab”?

In Pinecone, an **index** is a logical grouping of **namespaces** which is where the actual records are stored (i.e. indexed). Transparently to the end-user, the data in a namespace is dynamically partitioned into what we call “slabs”. A **slab** is made up of multiple files and contains all the information necessary to process a read operation for the data it contains.

Each slab contains multiple representations of the data inside it. These are all built simultaneously as the slab is being created:

- The raw vectors and metadata are stored separately
- The dense vectors are built into an ANN index. The algorithm used is dynamically chosen at the time of building the slab according to the amount of records to be indexed. Depending on the amount and size of the records (i.e. dimensionality), quantization is automatically performed and the records are stored as “projections”. This reduces the size of each index.
- If sparse vectors are present (either sparse-only records or hybrid records) then a separate sparse vector index is built.
- If required, a full-text search index is built
- Each metadata field gets built into its own bitmap index
- Finally, a manifest is created that describes all of the above. When a slab is read, the manifest tells the query executors (more on those later) how to handle it (i.e. where the files are located, which algorithm to use, etc).

Freshness Guarantees

Each namespace will be made up of dozens or even low hundreds of slabs at any given time and each one is built according to the data that it needs to contain.

These slabs are stored on blob storage and, together, represent the entire dataset for a namespace. At run-time, the slabs are cached on SSDs of a scalable set of query executors. When a query comes in, a scatter-gather operation is performed so that all slabs for a given namespace are processed in parallel. In this way, the resource pool can be dynamically expanded or contracted, and the latency of processing a query can be maintained as close to O(1) as possible regardless of total dataset size.

A key concept of this storage architecture is that each slab is **immutable** once written and is only ever read from. This means that read performance is never impacted by new writes coming into the system.

There is also no concept of Pinecone needing to "re-index" the data. As new data is written, new slabs are created. Over time as the number and size of slabs grow, they are merged (i.e. compacted) and atomically swapped into place. It is through this property that Pinecone optimises itself for both writes and reads: data coming into the system is persisted to disk and made available for reading/querying as quickly as possible. In the background, compaction runs to optimise the data structures for accuracy, latency and throughput.

Finally, as indexing strategies, algorithms and topologies improve and evolve, a users dataset can be rebuilt in the background without any downtime or re-ingestion.

Write Path: Fast, Consistent Data Ingestion

Pinecone processes writes in multiple phases. Each phase is optimized for different goals: acknowledgement speed, indexing throughput, optimisation for querying.

Write Acknowledgment

When you send a write request to Pinecone, we acknowledge it as soon as the operation reaches durable storage. Each write operation can contain up to 2MB of data in a single batch. The system writes the operation to a Write-Ahead Log (WAL) on S3 and immediately returns confirmation to your application.

This design delivers write latency under 100ms. We don't wait for indexing to complete before acknowledging the write. Larger batches have slightly higher latency, but you can batch hundreds of records per request depending on vector dimensions and metadata size.

For example, to saturate the 64MB/s S3 bandwidth limit, you would need to send 32 write operations per second with 300-500 vectors each—over 10,000 vectors per second. Most applications send smaller batches during steady-state streaming writes. This is acceptable because write latency remains consistently low.

Index Building

After the write operation reaches S3, the index builder processes it asynchronously. The index builder operates per namespace and uses a consistent hash ring to distribute work across multiple builder instances.

The index builder pulls up to 10,000 records at a time from the WAL and loads them into a memtable. The memtable flushes to disk when it reaches 10,000 records, a few MB in size, or after a few minutes since the last flush.

Records in the memtable are available for querying immediately, even before they flush to disk. This ensures that writes appear in search results within seconds regardless of index size.

Once the memtable flushes, the index builder picks up the next batch from the WAL and repeats the process. The index never needs to rebuild or reprocess the entire dataset to incorporate new data. This contrasts with technologies that require full rebuilds as indexes grow.

Updates appear in search results within seconds. The memtable provides immediate access to new data. Background processes handle persistence and optimization without affecting query latency or write throughput.

Slab Levels

Each memtable flush creates an L0 slab. L0 slabs are small and fast to create—they contain up to 10,000 records. As L0 slabs accumulate, the compaction process merges them into larger structures.

When 100 or more L0 slabs exist, compaction merges them into L1 slabs. Each L1 slab contains approximately 100,000 records. Multiple L1 slabs merge into L2 slabs containing approximately 1 million records each.

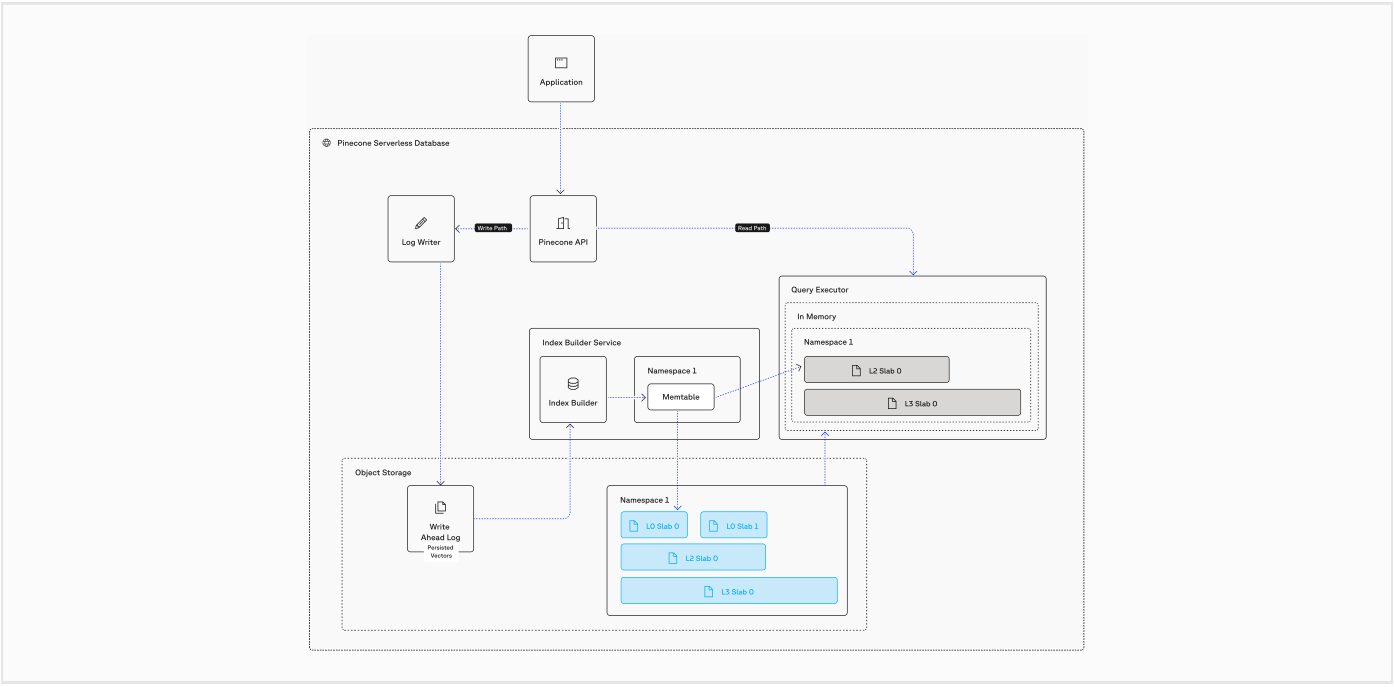
For on-demand and shared pool deployments, we limit the structure to L2 and cap slabs at roughly 10GB. This size ensures we have many reasonably sized files that can distribute across the resource pool. For Dedicated Read Nodes, we use an additional L3 level because each node dedicates to a single index and can handle larger files.

The slab level depends on dataset size, not compaction frequency. Compaction runs whenever necessary to compress and optimize vectors. As the dataset grows, more slabs at higher levels are created and eventually flow through to fewer, larger slabs at lower levels.

Compaction Process

Consider how data flows through the system:

1. Your first vector write goes to the WAL. The index builder picks it up, loads it into the memtable, and flushes it to disk as an L0 slab.
2. If no other writes occur, that single L0 slab serves all queries. This is unrealistic but illustrates the starting point.
3. As more records arrive, they also flush to disk as L0 slabs. More data means more L0 slabs.
4. When 100 L0 slabs accumulate, compaction gathers approximately 100,000 vectors from those slabs. It creates an L1 slab from those vectors, writes it to disk, notifies the query executors to cache it, and deletes the source L0 slabs.
5. If 150 L0 slabs existed when compaction started, 50 L0 slabs and 1 L1 slab remain after compaction. All queries now use those 51 slabs.
6. This repeats with the goal of keeping total slab count under 100. When more than 100 L1 slabs exist, compaction merges them into L2 slabs containing approximately 1 million vectors.
7. Now you might have 10 L0 slabs, 50 L1 slabs, and 3 L2 slabs. Queries use all 63 slabs.
8. While compaction runs, new writes continue creating L0 slabs. Compaction happens continuously as slabs accumulate.

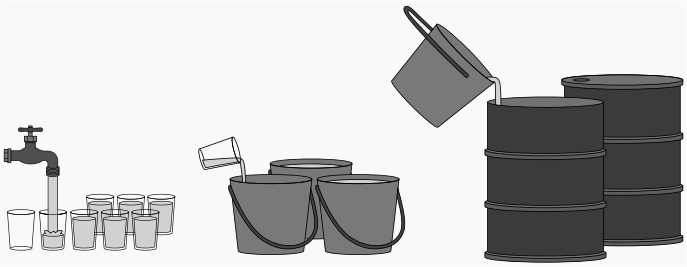


Analogy: Water, Cups, and Buckets

Imagine data flowing like water from a hose into small cups. As each cup fills, it's placed on the ground and the next cup starts to fill. When too many cups accumulate, someone pours them into a bucket and discards the empty cups. When too many buckets accumulate, someone pours them into a barrel and discards the empty buckets.

The water only flows from the hose into cups. Buckets take longer to fill. Barrels take even longer. The hose never connects directly to buckets or barrels, but the water reaches them so it can keep flowing.

In Pinecone, data is the water, cups are L0 slabs, buckets are L1 slabs, and barrels are L2 slabs. Data enters only through new L0 slabs. Compaction is the process that pours cups into buckets and buckets into barrels. The hose never connects directly to lower levels, but data reaches them through compaction.



Data Consistency: Tombstones and Version Control

Because slabs are immutable, a record can exist in multiple slabs simultaneously and this is expected. Consistency is maintained using tombstones.

When a record is written (upserted, updated, or deleted), the index builder checks if it already exists. This check performs a fast ID lookup against all slabs—not a vector search. If the ID exists, the index builder creates a tombstone for all slabs where that ID appears.

At query time, executors use the tombstone list for the namespace to exclude previous versions of the record. Only the latest version participates in vector search. Tombstones are inserted into the compaction process so that as we build the next slab, we remove outdated versions and their corresponding tombstones. This acts as garbage collection to prevent slab size from growing uncontrollably with repeated updates.

When a vector is deleted (whether by upsert, update, or delete operation), the tombstone flushes from the memtable into a tombstone file. These tombstones usually apply to multiple slabs, making them global tombstones.

We limit global tombstone files to 10. When 10 global tombstone files accumulate, we trigger a tombstone split operation. This takes each global tombstone file and generates individual tombstone files for each affected slab. The split operation doesn't remove tombstones—it redistributes them across files.

Tombstones are applied during slab compaction. All local tombstone files for the slabs being compacted are applied when creating the new slab. We also trigger single slab compaction when a slab has more than 15% of its vectors tombstoned. In this case we rebuild just that slab without the tombstoned records.

Dynamic algorithm choices per-slab

Because a dataset is automatically partitioned into multiple slabs, and because these slabs get merged/compacted in the background, Pinecone can optimise every slab and therefore every namespace for both high throughput writing as well as high throughput, accurate querying. New writes only need to create a new L0 slab which is very fast and makes them immediately available within the whole index. Queries continue to be served from active slabs until new ones are created and swap in. In this way, compaction can run for minutes or even hours without any impact to the performance nor accuracy of the system. It can use this time to perform complex quantization and

and use indexing algorithms that would otherwise be too slow to be part of the primary write path:

- The memtable lives in RAM and can be both written to and read from extremely quickly. It is limited to only 10,000 vectors at a time and so uses brute-force scanning which is perfectly accurate.
- Small slabs on disk (up to 100k) are also processed using a brute-force scan
- Up to 1 million vectors use an algorithm called ananas, our proprietary implementation of Fast Johnson-Lindenstrauss Transform (FJLT). At 1 million vectors, this algorithm can be tuned for fast writes and very fast reads.
- Larger slabs (over 1 million vectors) use Inverted File (IVF) indexing, which clusters vectors together. Each cluster is itself a small ananas index. This approach works better at larger scale and allows scanning only a small portion of the dataset by choosing a few IVF clusters based on the mathematical distance of the incoming query vector.
- The algorithm choice depends on slab size, not level. L0 slabs are always ananas because they're capped at 10,000 records. L1 and L2 slabs use ananas if small, but switch to IVF when larger. We determine this when the slab is built during compaction.
- We've recently introduced a new algorithm called Product-Quantized Fast Scan (PQFS), which replaces ananas in larger slabs with IVF clustering and provides higher recall with lower latency. This new algorithm is being rolled out to all customers incrementally without them having to make any changes.

This design demonstrates how much sophistication goes into building a Pinecone index. We don't force you to choose which algorithm to use for a dataset. You get the best approach for your data without manual tuning.

Read Path: Distributed Search and Result Merging

When processing a query, the data in all slabs as well as the memtable must be considered. The query is distributed to multiple executors in order to process all the slabs in parallel. Each executor is responsible for processing one or more slabs locally, and due to metadata filtering, not all data will be read every time.

Each query hits every slab, but the work distributes across our fleet of query executors. Each node, thread, and core handles only the slabs assigned to it. The filtering logic runs separately for each slab. This means that even for multi-billion vector indexes split into a few hundred slabs, the work parallelizes and latency stays relatively constant from small to extra-large scale.

Metadata Filtering

Metadata is indexed using roaring bitmaps that scan extremely quickly. If a metadata filter exists on the query, we decide between pre-filtering and post-filtering based on the filter's selectivity at query time.

If the filter matches a small proportion of vectors, we only scan the records that match the filter. If the filter matches a large proportion of records, we scan the index first and apply the filter during the scan. This varies between ananas and IVF slabs, but the principle remains: we choose the fastest approach based on the data.

Pinecone never performs traditional post-filtering, which finds top-k items without the filter and then applies it afterward. Traditional post-filtering can return fewer than k results. For sparse vector queries, we use pre-filtering by checking for metadata filter satisfaction before pushing items onto the heap. This increases the likelihood of returning the requested number of results.

Selective filters make queries faster. When you filter to a small subset, we scan fewer vectors. This contrasts with systems that slow down when filters apply. Our bitmap indexes and adaptive strategy maintain speed across different filter patterns.

Full-Fidelity Storage: No Forced Quantization

Most vector search technologies recommend or require reducing data size. They keep everything in RAM or can't update and read simultaneously. This approach uses quantization: either lower dimension vectors or reduced precision (from 32-bit down to 16-bit or 8-bit). The problem is that quantization loses accuracy.

Pinecone doesn't require you to quantize before storage. You can store vectors as large as you want at full fidelity. We automatically split your index into many files that distribute and process in parallel. Each scan requires less memory. We don't keep everything in RAM.

What we do put in RAM is highly optimized for space without losing accuracy. We use similar quantization methods internally, but our research team tunes them heavily. We constantly improve them and apply updates back to your indexes. This wouldn't be possible if you'd already reduced accuracy before storing vectors.

The compaction process is critical here. We can take as much processing time as needed to optimize lower-level slabs. More time means stronger algorithms. We can change and improve indexing structure and algorithms over time by re-compacting your original vectors.

Scaling Throughput: Dedicated Nodes

Pinecone offers two deployment models: On-Demand and Dedicated Read Nodes (DRN). Each model optimizes for different traffic patterns and cost structures.

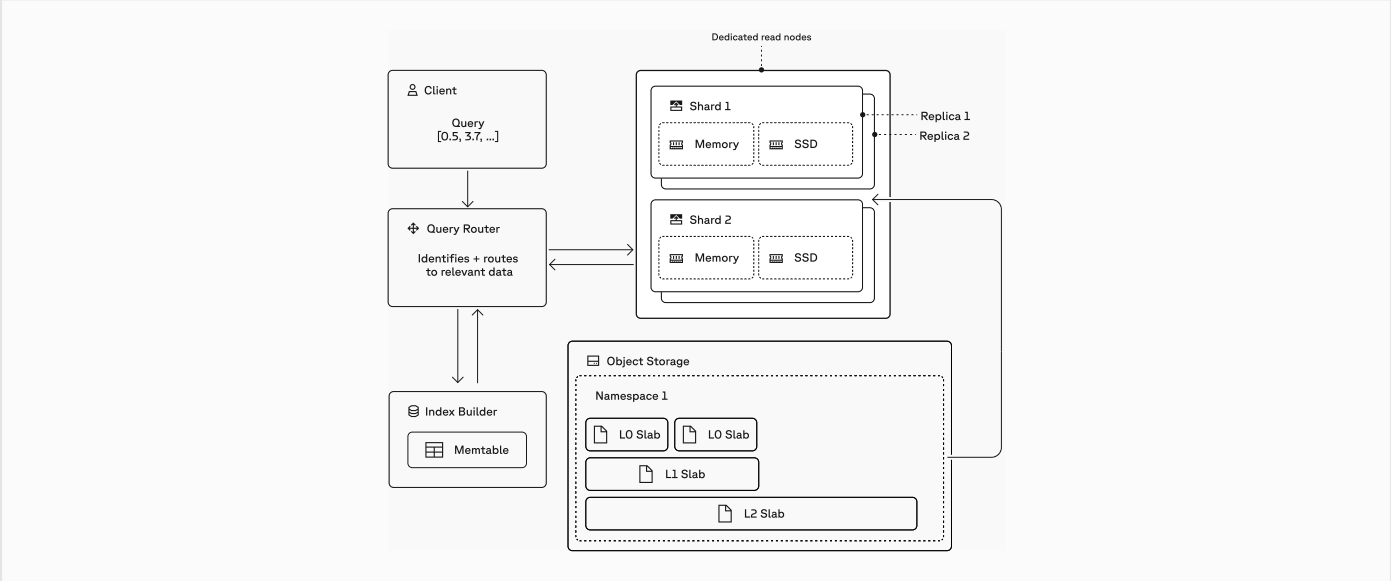
On-Demand

On-Demand indexes use shared infrastructure. Resources scale elastically based on usage. You pay per read unit consumed. The system handles variable traffic automatically.

On-Demand works well for workloads with inconsistent query patterns. Traffic can spike and drop without manual intervention. You don’t pay for idle capacity. However, high sustained traffic accumulates read unit charges quickly. The system may apply rate limiting during extreme bursts.

Dedicated Read Nodes

DRN indexes run on reserved infrastructure. You provision specific capacity for your index. Pricing is based on the number of shards and replicas you deploy, not query volume. Throughput scales independently via replicas. Charges are based on total shard count (the product of shard count and replica count).



A single large DRN index doesn't inherently penalize throughput. Meeting higher queries per second (QPS) is handled by increasing replicas via API or the Pinecone Console. DRN's flat-fee model suits sustained, high query traffic and avoids the read unit rate limiting that occurs with on-demand deployments. You pay the same cost whether you run 100 queries or 100,000 queries per hour. There is no read unit rate limiting.

For on-demand and shared pool deployments, we limit slabs to L2 and roughly 10GB. For Dedicated Read Nodes, we use an additional L3 level. Each node dedicates to a single index and can handle larger files.

Architectural Flexibility: Deployment Options

Pinecone supports multiple deployment patterns to balance cost and operational complexity.

Selective Migration

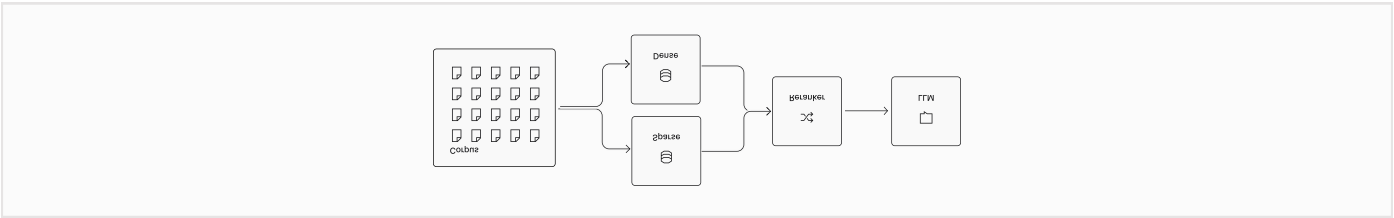
Move your most trafficked namespaces into their own DRN indexes. Less frequently accessed namespaces remain in on-demand indexes. This approach favors cost savings but increases architectural complexity.

Full Consolidation

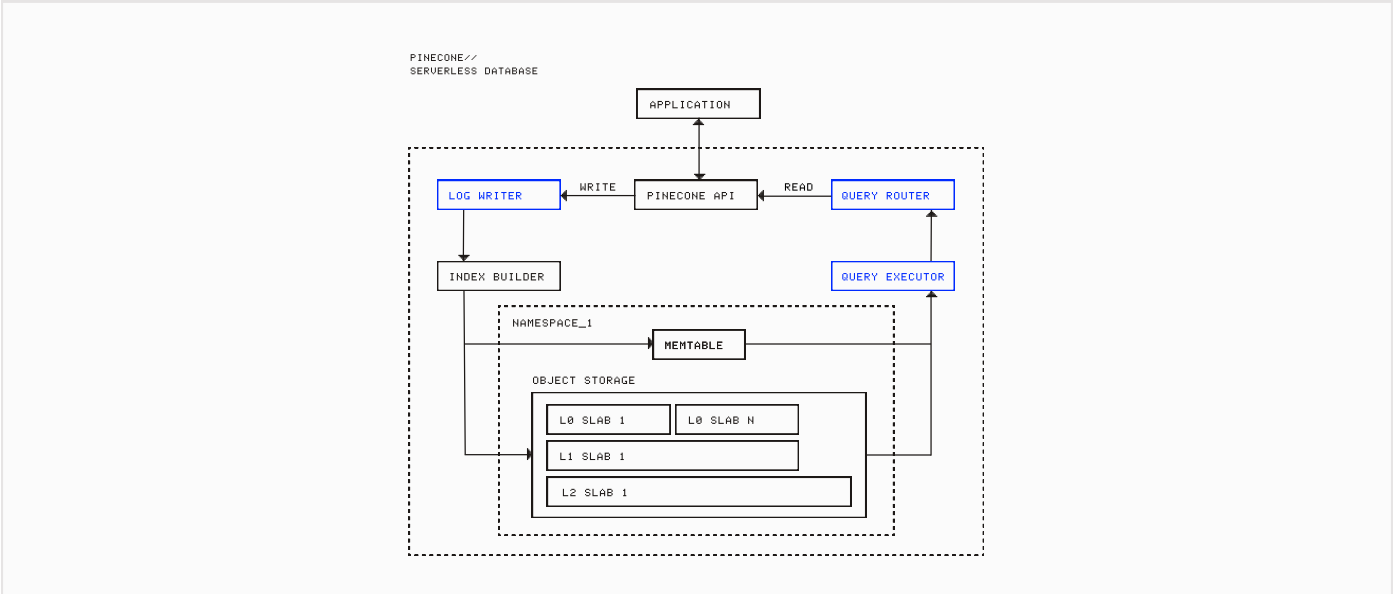
Combine all dense namespaces into one large namespace and move to DRN. Do the same for sparse indexes. This favors logistical simplicity but moderately increases costs. Short-term effort involves merging namespaces, which requires additional metadata filtering. Records must be written with an additional metadata field (for example, insurance_type). Reads must filter by this field on every request. The benefit is that you no longer query across multiple namespaces—you modify your metadata filter instead.

Hybrid Indexes

Combine sparse and dense vectors into a single hybrid DRN index. Everything exists in one namespace within one index. You no longer need to make rerank requests. However, sparse-only queries are not currently supported for hybrid indexes (dense-only queries are supported). We recommend testing with a small hybrid index to ensure accuracy meets your requirements before committing to this option.



System Components: High-Level Data Flow



An API gateway sits in front of the system and receives client requests. It routes them to either the read path or the write path.

In the read path, the query router is a significant component. It manages traffic to multiple query executors and the memtable, and merges results.

For the write path, when a write request arrives, it goes directly to the index builders using a consistent hash ring according to the namespace assignment for each builder. That index builder writes the operation to the write-ahead log (WAL) on blob storage and puts it into the memtable simultaneously.

Data Management Best Practices

Document Chunking

Most workflows break documents into chunks. Each chunk becomes a vector in Pinecone. When document content changes, the chunk count or text may differ. Best practice involves deleting all chunks for a document before upserting updated versions.

Use delete-by-metadata to remove all chunks associated with a document ID. Then upsert the new chunk set. This prevents orphaned or outdated chunks in search results.

Conditional Updates

Update-by-metadata changes metadata fields for vectors matching a filter. You can add or update fields but not remove them. This operation does not modify embeddings.

The operation completes silently if no vectors match the filter. Check version timestamps in metadata to decide whether to update or skip processing.

Namespace Organization

Namespaces isolate data within an index. Each namespace gets independent processing through the consistent hash ring. Index builders know which namespaces they handle.

You can organize by tenant, data type, or environment. Metadata filtering within a single namespace often provides simpler architecture than splitting across many namespaces.

Benefits of the Pinecone Architecture

No Accuracy Loss

Many systems require dimension reduction or precision loss to scale. Pinecone stores vectors at full fidelity. We apply quantization internally during compaction, using research-backed algorithms tuned for each workload.

You send full-precision vectors. We handle compression and optimization transparently. This preserves search quality while achieving efficient storage and fast queries.

Continuous Optimization

Our research team develops new algorithms continuously. When we improve indexing techniques, we apply them during compaction. Your

existing vectors benefit from better performance without re-ingestion or configuration changes.

This approach contrasts with static indexes that require manual rebuilding for improvements. Compaction lets us evolve the system while maintaining data availability.

Operational Simplicity

You do not tune ANN parameters or manage cluster configurations. Pinecone selects algorithms based on data characteristics. The system scales automatically through replicas and sharding.

This reduces expertise requirements for operating vector search in production. Teams focus on building applications instead of database internals.

Conclusion

Pinecone's architecture addresses the fundamental challenges in vector search through careful distributed systems design. The LSM-based slab system enables fast writes without sacrificing query performance. Adaptive metadata filtering makes searches faster when filters are selective. Full-fidelity storage preserves accuracy without forcing manual quantization.

We built these systems to eliminate the tradeoffs that plague traditional vector search. You don't choose between write speed and read performance. You don't tune parameters as your dataset grows. You don't reduce data fidelity to fit in RAM.

The system scales automatically. Writes acknowledge in under 100ms and appear in queries within seconds. Queries maintain low latency from millions to billions of vectors. This is the result of algorithmic innovation and distributed systems engineering working together.

For more technical documentation, visit docs.pinecone.io. To discuss your specific use case, contact our solutions team at pinecone.io/contact.